Rivard, S., Brière, Y. & Trudel, M-C. (À paraître)

Bases de données relationnelles

Dans : *Le développement des systèmes d'information : une méthode intégrée à la transformation des processus*, Annexe 7, 10 pages.

COPIE DE TRAVAIL, NE PAS REDISTRIBUER

BASE DE DONNÉES RELATIONNELLE : PRINCIPAUX CONCEPTS

Les bases de données constituent le cœur des systèmes d'information transactionnels. Les réponses à des questions telles que « Combien me doit un tel client ? Quand tel ou tel client a-t-il commandé pour la dernière fois? Quels sont les produits d'une commande ? Quels sont les produits d'un fournisseur » se trouvent dans les bases de données de l'organisation. En plus, lorsqu'on traite une transaction, on saisit et on modifie des données qui sont dans les bases de données.

Avec l'informatisation des processus d'affaires, les organisations se construisent d'énormes bases de données détaillant pratiquement tous les aspects de leurs opérations, des entrées et sorties d'argent en passant par les commandes faites par les clients, les niveaux d'inventaire et les heures travaillées par les employés. Ces bases de données constituent de véritables entrepôts qui, lorsqu'elles sont exploitées judicieusement, peuvent fournir aux entreprises un avantage compétitif important.

Par exemple, McKesson Corp, un distributeur de produits pharmaceutiques, conserve des données sur les commandes journalières qu'elle reçoit de ses clients pour ses 285 000 produits répartis dans ses 50 entrepôts, soit plus de 1,4 million de lignes de commandes différentes par jour. Selon les dernières statistiques, Mastercard gère près de 400 millions de transactions journalières. De son côté, Wal-Mart emmagasine depuis longtemps les données provenant des caisses enregistreuses dans ses quelque 10 800 magasins et se sert de cette information pour prédire le volume de chaque produit qui sera vendu dans chaque magasin.

Les entreprises se servent de leurs bases de données non seulement pour obtenir des réponses à des questions bien précises, mais aussi pour découvrir des patterns nouveaux. On utilise des logiciels spécialisés qui fouillent les bases de données et les analysent dans le but d'établir des associations entre les variables que l'on ne soupçonnait pas. Par exemple, les chaînes de supermarchés analysent couramment les données provenant des caisses enregistreuses afin de découvrir les profils d'achat des consommateurs. Les résultats des analyses permettent aux chaînes de revoir la disposition de leurs étalages pour qu'ils correspondent mieux aux comportements des consommateurs.

Étant donné l'importance des bases de données pour le bon fonctionnement des organisations, il est fondamental qu'elles soient bien conçues. La conception réussie d'une base de données est habituellement le résultat d'une collaboration étroite entre les professionnels des systèmes d'information et les utilisateurs éventuels. Dans ce texte nous verrons les concepts nécessaires pour bien comprendre comment organiser une base de données.

Nous présenterons ici les concepts nécessaires pour travailler efficacement avec des bases de données relationnelles. Bien que la théorie des bases de données relationnelles soit aujourd'hui très formalisée, faisant appel à plusieurs concepts mathématiques, nous décrirons ici les concepts de façon pratique en insistant sur ce qui est absolument nécessaire pour concevoir des bases de données relationnelles de qualité.

COPIE DE TRAVAIL, NE PAS REDISTRIBUER

Les principaux concepts reliés aux bases de données relationnelles que nous verrons dans ce texte sont :

1. Base de données;
2. Table;
3. Enregistrement;
4. Attribut;
5. Clé primaire ; ¸
6. Structure d'une table
7. Clé lointaine ;
8. Liens entre les tables ;
9. Diagramme de structure de la base de données.

1 Base de données

Une base de données relationnelle est composée d'un ensemble de tables reliées entre elles. Supposons une base de données pour un système de prise de commandes formée des quatre tables suivantes : CLIENT, PRODUIT, COMMANDE et DÉTAIL DES COMMANDES. La table PRODUIT contient des données sur les produits de l'entreprise, la table CLIENT, sur les clients de l'entreprise, la table COMMANDE, sur les commandes passées par les clients et la table DÉTAIL DES COMMANDES, sur les produits commandés dans chaque commande particulière d'un client.

Comme les tables sont reliées entre elles — par un attribut commun, comme nous le verrons plus tard —, il est facile de retrouver les données d'une table à partir de celles d'une autre. Par exemple, si on connaît le nom d'un client, il est alors possible d'obtenir la liste des commandes passées par ce client car la table CLIENT est reliée à la table COMMANDE par un attribut commun, *Numéro du client*.

En pratique, une base de données peut contenir plusieurs centaines de tables.

2 Table

La table dans une base de données relationnelle est l'objet logique qui contient les données. Chaque table est identifiée par un nom qui lui est propre. En principe, les données dans une table devraient décrire une entité de l'entreprise. Par exemple, la table PRODUIT de l'exemple précédent a la forme suivante :

COPIE DE TRAVAIL - NE PAS REDISTRIBUER

PRODUIT

Numéro du produit	Description du produit	Quantité en stock	Prix suggéré
P001	Écran 27 po	245	285,00
P003	Routeur	65	145,00
-	-	-	-
P999	-	-	-

Il n'y a aucune contrainte logique au nombre de lignes et de colonnes que peut posséder une table. Il n'est pas rare de voir en pratique des tables qui contiennent plusieurs millions de lignes.

3 Enregistrement

Chaque ligne de la table est aussi appelée un enregistrement. Un enregistrement est défini par l'ensemble des valeurs composant la ligne. Par exemple, l'ensemble des valeurs (P001 ; Écran 27 po ; 245 ; 285,00) définit l'enregistrement du produit P001.

Dans une base de données relationnelle, tous les enregistrements d'une table sont uniques, c'est-à-dire qu'il n'y en a pas deux qui ont exactement le même ensemble de valeurs et ils sont placés dans la table les uns à la suite des autres, sans aucun ordre particulier.

4 Attribut

Chaque enregistrement d'une table est formé par un certain nombre d'éléments d'information que l'on appelle attributs ou champs. Par exemple, dans la table PRODUIT, les enregistrements sont décrits par quatre attributs : *Numéro du produit*, *Description du produit*, *Quantité en stock* et *Prix suggéré*. Les attributs sont présentés en colonne. Il faut faire la distinction entre l'attribut (p. ex. *Description du produit*) et la valeur prise par l'attribut pour un enregistrement particulier (p. ex. écran 27 po).

Chaque attribut est identifié par un nom qui lui est propre. L'ensemble des valeurs possibles que peut prendre un attribut s'appelle le domaine de l'attribut. Dans notre exemple, le domaine de l'attribut *Numéro du produit* est l'ensemble de toutes les valeurs commençant par la lettre P suivie de trois chiffres.

La décision de conserver ou non un attribut dans la base de données appartient au concepteur. Si l'on respecte les principes de conception des bases de données, il n'y a pas de limite logique au nombre d'attributs que peut comprendre une table.

Chaque attribut possède un certain nombre de propriétés que le concepteur de la base de données doit déterminer. Ces propriétés sont en fait des contraintes qui permettent d'assurer l'intégrité des données. Si la valeur d'un attribut ne respecte pas les propriétés spécifiées au préalable lors de la création de la base de données, alors le système de gestion de bases de données refusera de l'entrer dans la table.

Les Systèmes de Gestion des Base de Données (SGBD) n'offrent pas tous exactement les mêmes fonctionnalités en ce qui a trait aux propriétés des attributs, mais voici les principales que l'on retrouve

COPIE DE TRAVAIL - NE PAS REDISTRIBUER

chez la plupart d'entre eux :

Type de valeur. Avec cette contrainte, on force les valeurs d'un attribut à être d'un certain type. Les principaux types sont : texte, alphanumérique, entier, réel, monétaire, date/heure, booléen. Encore une fois, les types exacts peuvent varier d'un SGBD à l'autre. Si la valeur saisie pour un attribut ne correspond pas au type de valeur associé à cet attribut lors de la création de la base de données, alors le système de gestion de base de données refusera de l'entrer dans la table. Par exemple, les types de données pour les attributs Numéro du produit, Description du produit, Prix suggéré et Quantité en stock sont respectivement : alphanumérique, texte, monétaire et entier.

Obligatoire. Le concepteur peut décider de rendre un attribut obligatoire. Lorsqu'un attribut est défini comme obligatoire, il doit en tout temps posséder une valeur. Si un attribut est obligatoire, alors il n'est possible de créer un enregistrement dans la base de données que si l'on saisit une valeur pour l'attribut en question. Comme c'est une contrainte très restrictive, le concepteur ne doit rendre un attribut obligatoire que lorsque c'est absolument nécessaire pour préserver l'intégrité des données. Trop d'attributs obligatoires peuvent nuire au bon fonctionnement du système d'information. Par exemple, il est important dans un système de prise de commandes de pouvoir créer un client dans la base de données même si l'on ne possède pas toutes les données à son sujet. De même, dans beaucoup de systèmes, un enregistrement est créé à un certain moment dans le temps sachant que l'information pour certains attributs ne sera disponible que plus tard. Lorsqu'on ne connaît pas la valeur pour un attribut et qu'on le laisse vide, on dit alors qu'il prend la valeur NULLE.

Le tableau suivant présente les principaux types de valeurs pour les attributs d'une base de données.

Type de valeur	Description
Texte	N'importe quel caractère incluant lettres, chiffres et caractères spéciaux.
Alphanumérique	Valeur composée de chiffres ou de lettres qui sert de numéro d'identification (code permanent d'étudiant, numéro d'assurance maladie, numéro de permis de conduire, code postal, numéro de téléphone). Il est possible de trier les valeurs alphanumériques par ordre ascendant ou descendant. Habituellement, les valeurs alphanumériques ont un format bien précis qu'il est possible de définir. Par exemple, le code postal canadien peut être décrit de la façon suivante : 6 caractères commençant par une lettre, alternant par la suite entre lettre et chiffre et ne contenant aucun caractère spécial. Il est préférable de choisir le type alphanumérique, au lieu du type entier, pour des numéros composés uniquement de chiffres tels que le numéro de téléphone sur lesquels on n'est pas censé effectuer des calculs.
Entier	Nombre entier.
Réel	Nombre réel.
Monétaire	Valeur monétaire.
Date/heure	Valeur qui représente la date ou l'heure. La plupart des SGBD offrent de nombreux formats pour les dates et l'heure.
Booléen	Type qui ne peut prendre que deux valeurs : vrai ou faux

Unique. Il est parfois nécessaire de s'assurer que les valeurs prises par un attribut sont différentes pour tous les enregistrements dans la table ; en d'autres mots que les valeurs de l'attribut sont uniques pour chaque enregistrement. Par exemple, l'attribut Numéro du client dans une table CLIENT doit être défini comme étant unique. Il ne doit pas être possible de créer deux clients différents avec le même numéro de client.

COPYRIGHT - NE PAS REDISTRIBUER

Intervalle de validité. Dans certains cas, il peut être important de s'assurer que les valeurs prises par un attribut sont comprises dans un intervalle de validité. Par exemple, un attribut Note de l'étudiant devra avoir ses valeurs entre 0 et 100.

Intégrité référentielle. Cette dernière condition permet de s'assurer, avant de créer un enregistrement dans une table, que la valeur d'un attribut existe dans une autre table. Par exemple, dans une table COMMANDE, on ne voudrait entrer des commandes que pour des clients qui existent dans la table CLIENT. Pour s'en assurer, il faut que la valeur de l'attribut Numéro du client dans la table COMMANDE existe dans les valeurs prises par l'attribut Numéro du client dans la table CLIENT.

5 Clé primaire

Le concept de clé est fondamental à la compréhension des bases de données. On dit d'un attribut qu'il est clé dans une table s'il permet d'identifier de façon unique les enregistrements de celle-ci. Un attribut clé doit avoir les deux propriétés suivantes : il doit être unique et obligatoire ; cette dernière propriété est nécessaire pour assurer l'unicité des enregistrements. En effet, si l'attribut n'est pas obligatoire, plusieurs enregistrements peuvent prendre la valeur nulle. Des exemples d'attribut clé seraient le Numéro d'employé dans une table EMPLOYÉ, le Numéro de client dans une table CLIENT, le Numéro de commande dans une table COMMANDE.

Lorsque deux ou plusieurs attributs d'une même table peuvent jouer le rôle de clé, alors on dit que ces attributs sont des candidats pour devenir clé primaire de la table. Par exemple, une table qui contient les attributs Numéro d'employé et Numéro d'assurance sociale possède deux candidats pour devenir clé primaire de la table.

Pour assurer l'unicité des enregistrements, il est parfois nécessaire de combiner plusieurs attributs. Lorsque la clé primaire est composée de plusieurs attributs, on parle de clé primaire composée ou concaténée. Les attributs composant la clé primaire doivent nécessairement avoir les deux propriétés suivantes : chaque attribut membre de la clé primaire doit être obligatoire et les valeurs pour les deux attributs pris ensemble doivent être uniques. Il est important de remarquer que dans le cas d'une clé primaire concaténée, chaque attribut pris individuellement n'est pas unique.

Prenons la table suivante décrivant le détail des commandes.

DÉTAIL DES COMMANDES

Numéro de la commande	Numéro du produit	Quantité commandée
C001	P001	10
C001	P004	25
C002	P001	45
C003	P004	15
C003	P001	10
C003	P008	12

Dans cette table, on remarque que la commande C001 comprend deux produits P001 et P004, la commande C002 un produit P001 et la commande C003 trois produits P001, P004 et P008. En d'autres mots, chaque commande peut contenir plusieurs produits et chaque produit peut se retrouver dans différentes commandes. On remarque aussi que ni l'attribut Numéro de la

COPIE DE TRAVAIL NE PAS REDISTRIBUER

commande, ni l'attribut Numéro du produit ne permettent d'identifier de façon unique chacun des enregistrements. En effet, on note que la valeur C001 de l'attribut Numéro de la commande est répétée plusieurs fois dans la table ; de même, la valeur P004 de l'attribut Numéro du produit est répétée deux fois. Donc pour identifier de façon unique chacun des enregistrements de la table, il faut combiner les deux attributs, **Numéro de la commande** et **Numéro du produit**. Les deux attributs considérés simultanément forment un identificateur unique.

6 Structure d'une table

La structure d'une table est définie par l'ensemble des attributs composant la table. On utilise aussi parfois l'expression schéma d'une table. La structure de la table peut être représentée graphiquement de plusieurs façons :

1. Le nom de la table est placé en avant des parenthèses qui contiennent le nom des attributs séparé par une virgule.

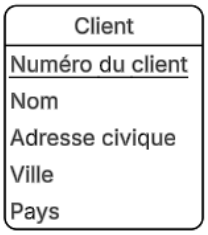
   CLIENT (Numéro du client, Nom, Adresse civique, Ville, Pays)

2. Le nom de la table est placé au-dessus d'un tableau horizontal contenant le nom des attributs.

   CLIENT

Numéro du client	Nom	Ville	Adresse civique	Pays

3. Le nom de la table est placé au-dessus d'un tableau vertical contenant le nom des attributs. Cette représentation est la plus utilisée.



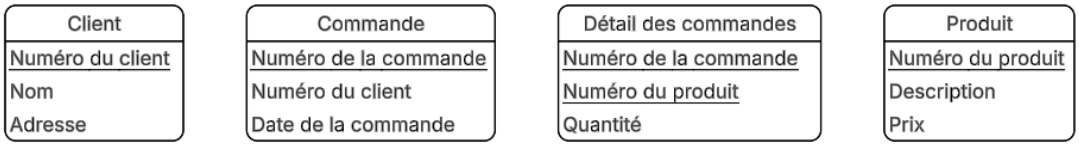
On remarque que, peu importe la représentation graphique, la clé primaire est indiquée en soulignant le ou les attributs faisant partie de la clé.

Le schéma de la table représente donc la définition générale de la table tandis qu'un enregistrement est une instance particulière du schéma de la table. Créer un enregistrement consiste en fait à assigner une valeur particulière à chacun des attributs du schéma de la table. Évidemment, les valeurs doivent respecter les contraintes spécifiées.

7 Clé lointaine

On dit qu'un attribut d'une table est une clé lointaine lorsqu'il est une clé primaire dans une autre table. Par exemple, si l'on a une base de données composée des quatre tables suivantes :

COPIE DE TRAVAIL, NE PAS REDISTRIBUER



Numéro du client dans la table COMMANDE est une clé lointaine car Numéro du client est clé primaire de la table CLIENT ; de même Numéro de la commande et Numéro du produit dans la table DÉTAIL DES COMMANDES sont des clés lointaines car ces deux attributs sont respectivement clés primaires des tables COMMANDE et PRODUIT.

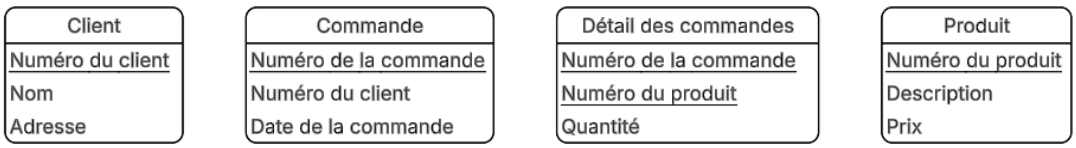
Notez que Numéro de la commande et Numéro du produit ne sont ni l'un ni l'autre clé primaire de la table DÉTAIL DES COMMANDES, mais font plutôt partie de la clé primaire concaténée (Numéro de la commande + Numéro du produit).

Il faut remarquer que la clé lointaine n'est pas unique car on retrouve la même valeur plusieurs fois dans la table. Comme la clé lointaine sert à faire le lien avec l'autre table, elle doit absolument respecter la contrainte d'intégrité référentielle, c'est-à-dire que la valeur que prend la clé lointaine doit exister dans la table où l'attribut est clé primaire. En effet, si la clé lointaine pouvait prendre des valeurs qui n'existent pas dans la table où cet attribut est clé primaire, il serait alors impossible de créer un lien entre les deux tables. En d'autres mots, il serait possible d'avoir des commandes qui n'appartiennent à aucun client.

Cependant, la clé lointaine n'est pas nécessairement obligatoire. Elle peut, suivant la situation, prendre la valeur nulle. Par exemple, on pourrait souhaiter inscrire un produit dans la base de données sans qu'on lui ait encore affecté un fournisseur. Dans ce cas, l'attribut Numéro du fournisseur prend la valeur nulle dans la table PRODUIT. Cependant, la clé lointaine Numéro du client dans la table COMMANDE devra, quant à elle, toujours posséder une valeur car on ne voudrait pas qu'une commande soit inscrite dans la base de données sans qu'il y ait un client qui lui soit associé.

8 Liens entre les tables

Nous avons mentionné plus haut qu'une base de données est composée de plusieurs tables reliées entre elles. Les liens entre les tables dans le modèle relationnel se font à l'aide d'attributs communs entre les tables. Le lien s'établit entre la clé d'une table et le même attribut qui se retrouve dans une autre table ; en d'autres mots, entre la clé primaire d'une table et la clé lointaine correspondante. Prenons, par exemple, la base de données composée des quatre tables suivantes :



Cette base de données comprend les liens suivants :

- entre CLIENT et COMMANDE, Numéro du client étant clé primaire dans la table CLIENT et apparaissant dans la table COMMANDE ;
- entre COMMANDE et DÉTAIL DES COMMANDES, Numéro de la commande étant clé primaire dans

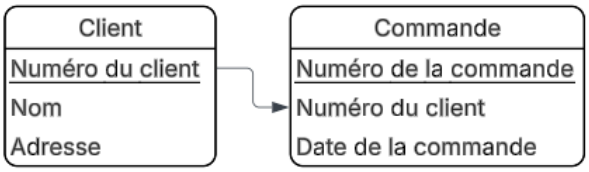
GUIDE DE TRAVAIL NE PAS REDISTRIBUER

la table COMMANDE et apparaissant dans la table DÉTAIL DES COMMANDES ;

- entre PRODUIT et DÉTAIL DES COMMANDES, Numéro du produit étant clé primaire dans la table PRODUIT et apparaissant dans la table DÉTAIL DES COMMANDES.

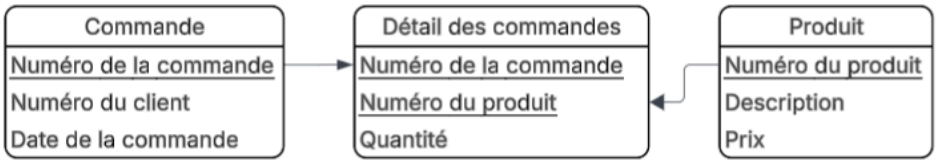
La détermination des liens entre les tables se fait donc de façon très mécanique. Il s'agit tout simplement d'identifier la clé primaire d'une table et de chercher si cette clé primaire apparait dans une autre table de la base de données.

Lorsqu'il y a un lien entre deux tables de la base de données, on trace une flèche entre les deux attributs communs pour indiquer le lien entre les deux tables. Par exemple, entre les tables CLIENT et COMMANDE, on peut tirer le lien suivant :



On remarque une flèche à une des extrémités du lien, qui pointe vers la clé lointaine. Cela signifie que le numéro du client se trouvant dans la table CLIENT est susceptible de se retrouver plus d'une fois dans la table COMMANDE. En termes plus concrets, cela signifie tout simplement qu'un client peut passer plusieurs commandes. C'est ce que l'on appelle un lien 1 @ N. La plupart des liens dans une base de données relationnelles sont de type 1 @ N. On retrouve parfois des liens 1 @ 1, c'est-à-dire qu'une valeur de la clé primaire ne peut se retrouver qu'une seule fois dans l'autre table. Lorsqu'on a un lien 1 @ 1, on ne met pas de flèche du côté de la clé lointaine car cet attribut est également unique dans la table où il est clé lointaine. Le concept de liens N @ M (plusieurs à plusieurs) n'existe pas dans une base de données relationnelles.

Il est important de remarquer que, dans le cas d'une clé concaténée, le lien entre les tables s'établit en prenant la clé dans son ensemble et non pas entre les attributs pris individuellement. Donc dans la table DÉTAIL DE LA COMMANDE, la clé est la fusion de *Numéro de la commande + Numéro du produit*.



Une clé devrait-elle être significative ?

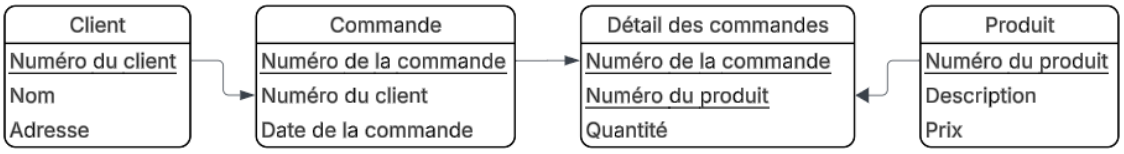
La discussion précédente fait bien ressortir le rôle très important que joue la clé primaire dans une base de données relationnelle. En effet, elle permet non seulement d'identifier de façon unique chaque enregistrement d'une table, mais aussi de faire le lien entre les différentes tables de la base de données. Donc la valeur d'une clé primaire se trouve répétée plusieurs fois dans les autres tables de la base de données à titre de clé lointaine. Par exemple, la valeur de la clé primaire Numéro du client d'un client particulier apparait 233 fois dans la table COMMANDE si celui-ci a passé

COPIE DE TRAVAIL NE PAS REDISTRIBUER

233 commandes. Il faut donc éviter à tout prix d'avoir à modifier la valeur d'une clé une fois que celle-ci a été créée. Dans notre exemple, il faudrait tout d'abord modifier le numéro du client dans la table CLIENT puis modifier 233 fois ce même numéro du client dans la table COMMANDE. Si par malheur une occurrence est oubliée, les données dans la base de données ne sont plus exactes. C'est pour cette raison qu'il est préférable de ne pas utiliser comme clé un code dont certaines composantes ont des significations bien précises, tel le numéro d'assurance maladie émis par le gouvernement du Québec qui contient entre autres la date de naissance de l'individu, même si ce code est unique. Il est toujours possible qu'une erreur se glisse lors de la création du code et que l'individu revienne plus tard demandant de rectifier la situation ; il se pourrait aussi que des clients n'ait pas de numéro d'assurance maladie, ils ne pourraient donc pas passer de commande dans notre organisation. Il est donc recommandé d'utiliser un numéro automatique généré par le SGBD comme valeur de la clé primaire lorsque l'enregistrement est créé pour la première fois et d'éviter de donner toute signification à cette valeur autre que l'identification unique des enregistrements de la table. Les éléments significatifs deviennent tout simplement des attributs de la table. De cette façon, on s'assure que l'on n'aura jamais à modifier la valeur d'une clé.

9 Diagramme de structure de base de données (DSBD)

Le diagramme de structure de données (DSD) ou diagramme de structures de base de données (DSBD) sert à représenter graphiquement le schéma de la base de données. Plus précisément, il présente les attributs et la clé de chaque table ainsi que les liens entre les tables.



Pour compléter la représentation de la base de données, on doit construire un tableau par table qui décrit les contraintes pour chacun des attributs. L'ensemble de ces tableaux forment une partie du dictionnaire des données de la base de données.

COPIE DE TRAVAIL NE PAS REDISTRIBUER

Table CLIENT					
Attribut	**Type**	**Obligatoire**	**Unique**	**Intégrité référentielle**	**Intervalle de validité**
Numéro du client	Alphanumérique	Oui	Oui		Entre 00 000 et 99 999
Nom	Texte	Oui			
Adresse	Texte				

Table PRODUIT					
Attribut	**Type**	**Obligatoire**	**Unique**	**Intégrité référentielle**	**Intervalle de validité**
Numéro du client	Alphanumérique	Oui	Oui		Entre 00 000 et 99 999
Description	Texte	Oui			
Prix	Monétaire				> 0

Table COMMANDE					
Attribut	**Type**	**Obligatoire**	**Unique**	**Intégrité référentielle**	**Intervalle de validité**
Numéro de la commande	Alphanumérique	Oui	Oui		Entre 00 000 et 99 999
Numéro du client	Alphanumérique	Oui		Doit exister dans la table CLIENT	Entre 00 000 et 99 999
Date de la commande	Date/heure				

Table DÉTAIL DES COMMANDES					
Attribut	**Type**	**Obligatoire**	**Unique**	**Intégrité référentielle**	**Intervalle de validité**
Numéro de la Commande	Alphanumérique	Oui	Oui	Doit exister dans la table COMMANDE	Entre 00 000 et 99 999
Numéro du produit	Alphanumérique	Oui	Oui	Doit exister dans la table PRODUIT	Entre 00 000 et 99 999
Quantité	Entier				> 0

COPIE DE TRAV

Le travail du concepteur de bases de données consistera donc à identifier les attributs à conserver dans la base de données, à les organiser dans différentes tables, à définir les contraintes sur chaque attribut, à déterminer les clés primaires de chaque table, à indiquer les liens entre les tables et, finalement, à tracer le DSBD.